

Menu-Driven C Program Using `while(1)`

1 Introduction

A menu-driven C program is a program that displays a list of options for the user to choose from, and then performs actions based on the user's choice. It typically involves loops, conditionals, and user input. Here's a step-by-step guide on how to implement a basic menu-driven program in C.

2 Steps to Implement a Menu-Driven Program

2.1 1. Display the Menu

The first step in creating a menu-driven program is to display a list of options for the user to choose from. This is typically done using the `printf` function in C. The options may include various operations or tasks that the user can perform.

```
printf("Menu:\n");
printf("1. -Add-Numbers\n");
printf("2. -Subtract-Numbers\n");
printf("3. -Multiply-Numbers\n");
printf("4. -Divide-Numbers\n");
printf("5. -Exit\n");
```

The program should display these options to the user, providing them with a clear set of choices.

2.2 2. Take User Input

After displaying the menu, the next step is to ask the user to enter their choice. This can be done using the `scanf` function in C. The user's input is stored in a variable, which will later be used to determine what action to take.

```
printf("Enter your choice:-");
scanf("%d", &choice);
```

Here, the program asks for a numerical input and stores it in the variable `choice`. This input will be used to decide which action the program will perform.

2.3 3. Perform the Action

Based on the user's input, the program performs the corresponding operation. This can be done using a `switch-case` or an `if-else` statement. Depending on the value of `choice`, the program will call a specific function to carry out the operation.

```

switch (choice) {
    case 1:
        addNumbers();
        break;
    case 2:
        subtractNumbers();
        break;
    case 3:
        multiplyNumbers();
        break;
    case 4:
        divideNumbers();
        break;
    case 5:
        printf("Exiting program...\n");
        return 0;
    default:
        printf("Invalid choice! Please try again.\n");
}

```

This structure allows the program to execute the correct action based on the user's choice. If the user enters a choice outside the valid range (e.g., 1–5), the program informs the user that the input is invalid.

2.4 4. Loop Until User Chooses to Exit

The program should continue to display the menu and prompt for user input until the user chooses the exit option. This can be accomplished by using a loop. In this case, an infinite loop, such as `while(1)`, is used to ensure the menu keeps appearing until the user selects the exit option (e.g., 5).

2.5 Using while(1) Loop for Continuity

The entire process of displaying the menu, taking user input, and performing the action is enclosed inside an infinite loop, created by the `while(1)` condition. As long as the user does not choose the exit option, the program will keep calling the function to get input, perform the corresponding operation, and then print the menu again. This process repeats until the exit option is selected.

```

while(1) {
    // Display the menu
    printf("Menu:\n");
    printf("1. Add-Numbers\n");
    printf("2. Subtract-Numbers\n");
    printf("3. Multiply-Numbers\n");
    printf("4. Divide-Numbers\n");
    printf("5. Exit\n");

    // Get user input
    printf("Enter your choice:-");
    scanf("%d", &choice);

    // Perform the action based on input
    switch (choice) {
        case 1:
            addNumbers();
            break;

```

```

        case 2:
            subtractNumbers();
            break;
        case 3:
            multiplyNumbers();
            break;
        case 4:
            divideNumbers();
            break;
        case 5:
            printf("Exiting program...\n");
            return 0; // Exit the program
        default:
            printf("Invalid choice! - Please try again.\n");
    }
}

```

In this structure, the program will continually display the menu, ask for user input, perform the corresponding task, and then show the menu again. This cycle continues until the user chooses option 5 (Exit).

3 Full Code Example

Here is the full code that implements the menu-driven program:

```

#include <stdio.h>

// Function prototypes
void addNumbers();
void subtractNumbers();
void multiplyNumbers();
void divideNumbers();

int main() {
    int choice;
    while(1) {
        // Display the menu
        printf("Menu:\n");
        printf("1.-Add-Numbers\n");
        printf("2.-Subtract-Numbers\n");
        printf("3.-Multiply-Numbers\n");
        printf("4.-Divide-Numbers\n");
        printf("5.-Exit\n");

        // Get user input
        printf("Enter your choice:-");
        scanf("%d", &choice);

        // Perform the action based on input
        switch (choice) {
            case 1:
                addNumbers();
                break;
            case 2:
                subtractNumbers();

```

```

        break;
    case 3:
        multiplyNumbers();
        break;
    case 4:
        divideNumbers();
        break;
    case 5:
        printf("Exiting program...\n");
        return 0; // Exit the program
    default:
        printf("Invalid choice! Please try again.\n");
    }
}
return 0;
}

// Function implementations
void addNumbers() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Sum: %d\n", a + b);
}

void subtractNumbers() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Difference: %d\n", a - b);
}

void multiplyNumbers() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    printf("Product: %d\n", a * b);
}

void divideNumbers() {
    int a, b;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    if (b != 0) {
        printf("Quotient: %.2f\n", (float)a / b);
    } else {
        printf("Error! Division by zero.\n");
    }
}
}

```

4 Note:

In a menu driven program we try to implement each functionality as a function which is called based on the input of the user.